

Chapitre 6 : Organiser un programme

Par **Robert DIASSÉ**, Ingénieur Logiciel et Data

> **Cours** : Python avancé · **Fil rouge** : le registre des colis de Yonnee Express · **Version** : 2026-10-23

Fiche du chapitre

Objectifs	A la fin du chapitre, l'apprenant sait : 1) découper un programme en **modules** et les importer de trois façons ; 2) expliquer ce que fait Python lors d'un `import`, et le rôle de `if __name__ == "__main__":` ; 3) diagnostiquer les erreurs d'importation, dont le fichier qui **masque** un module de Python et l'importation **circulaire** ; 4) regrouper des modules en **paquet** et utiliser les importations relatives ; 5) créer un **environnement virtuel**, y installer un paquet avec `pip` et noter les dépendances dans `requirements.txt` ; 6) remplacer les `print` de diagnostic par un **journal** (`logging`) avec des niveaux, à l'écran et dans un fichier ; 7) expliquer pourquoi la suite logique est la programmation orientée objet.
Prérequis	Chapitres 1 à 5 ; votre dépôt `yonnee-express`
Durée	🟢 Débutant : 6 h · 🔵 Standard : 4 h 30 · 🟣 Expert : 3 h (atelier compris)
Matériel	Python 3.12 ou plus récent (vérifié avec Python 3.13), VS Code, terminal, une connexion Internet (pour `pip`)
Livrable	L'atelier 6 : l'outil Yonnee Express réorganisé en paquet, avec son environnement virtuel et son journal, publié sur GitHub

Comment lire ce chapitre

- **Noyau** : obligatoire pour tous les niveaux.
- 🟢 **Débutant** : explications pas à pas.
- 🔵 **Standard** : le niveau de référence.
- 🟣 **Expert** : approfondissements.

Créez un dossier `essais-ch6` . **Tous** les fichiers des sections 2 à 4 vont dans ce même dossier, et vous lancez les programmes depuis ce dossier (dans le terminal de VS Code : `cd essais-ch6`).

---

1. Le problème : un seul fichier, ou quatre copies

Regardez votre dépôt `yonnee-express` : `registre.py` (atelier 2), `statistiques.py` (atelier 3), `rapports.py` (atelier 4), `grand_registre.py` (atelier 5). Le tuple nommé `Livraison`, la lecture du CSV et les tarifs y sont **recopiés** plusieurs fois. Si le tarif de Pikine change, il faut le corriger à quatre endroits, et en oublier un donne des résultats faux sans aucune erreur.

La solution des programmeurs : **un endroit pour chaque chose**. Les tarifs dans un fichier, la lecture dans un autre, les filtres dans un troisième, et chaque programme **importe** ce dont il a besoin. Ce chapitre apprend à le faire proprement, puis à gérer ce que le programme utilise de l'extérieur (des paquets installés) et ce qu'il raconte pendant qu'il tourne (un journal).

2. Les modules

2.1 Un module, c'est un fichier

Un **module** est simplement un fichier `.py`. Créez `tarifs.py` :

```
"""Les tarifs de Yonnee Express."""
print("chargement du module tarifs")

TARIFS = {"Médina": 1000, "Plateau": 1500, "Yoff": 2000, "Parcelles": 2000, "Pikine": 2500}
SUPPLEMENT_LOURD = 500


def frais(quartier, poids):
    """Les frais de livraison d'un colis."""
    montant = TARIFS[quartier]
    if poids > 5:
        montant += SUPPLEMENT_LOURD
    return montant
```

Le `print` du début n'est là que pour voir **quand** le module est exécuté. Puis, dans le même dossier, `essai1.py` :

```
import tarifs

print(tarifs.frais("Yoff", 7))
print(tarifs.TARIFS["Pikine"])
print(tarifs.__name__, tarifs.__doc__)
import tarifs
print("fin")
```

```
chargement du module tarifs
2500
2500
tarifs Les tarifs de Yonnee Express.
fin
```

Morceau	Ce qu'il veut dire
`import tarifs`	Cherche le fichier `tarifs.py`, l'**exécute** une fois, et crée le nom `tarifs` qui donne accès à tout ce qu'il a défini. On écrit le nom **sans** `.py`
`tarifs.frais(...)`, `tarifs.TARIFS`	Le point : « la fonction `frais` **du module** `tarifs` »
`tarifs.__name__`	Le nom du module : `"tarifs"`
`tarifs.__doc__`	La docstring de la première ligne du fichier
Le second `import tarifs`	N'affiche **rien** : un module n'est exécuté qu'à sa première importation. Ensuite, Python réutilise celui qu'il a déjà en mémoire

🟢 Retenez la première ligne du tableau : importer un module, c'est **exécuter** son fichier. Tout ce qui est écrit au premier niveau du module (pas dans une fonction) s'exécute à l'importation : définitions, mais aussi `print`, calculs, ouverture de fichiers... C'est le problème que règle la section 3.

2.2 Trois façons d'importer

Créez `essai2.py` :

```python
from tarifs import frais, TARIFS
import tarifs as t

print(frais("Médina", 2))
print(sorted(TARIFS))
print(t.SUPPLEMENT_LOURD)
print(frais is t.frais)
```

```
chargement du module tarifs
1000
['Médina', 'Parcelles', 'Pikine', 'Plateau', 'Yoff']
500
True
```

Écriture	Ce qu'on obtient	Quand l'utiliser
`import tarifs`	Le nom `tarifs` ; on écrit `tarifs.frais(...)`	Le cas général : on voit d'où vient chaque nom
`from tarifs import frais, TARIFS`	Directement les noms `frais` et `TARIFS`	Quelques noms très utilisés

Écriture	Ce qu'on obtient	Quand l'utiliser
`import tarifs as t`	Le module sous un nom plus court	Les noms longs ou consacrés : `import pandas as pd`

`frais is t.frais` vaut `True` : c'est la **même** fonction, qu'on y accède d'une façon ou de l'autre (et le module n'a été chargé qu'une fois).

⚠️ Il existe aussi `from tarifs import *` (« tout importer »). Évitez-le : on ne sait plus d'où vient chaque nom, et un nom importé peut en écraser un autre sans prévenir.

> 🖍️ **Essayez** : dans un fichier à part, `import tarif` (sans s) ; puis `from tarifs import fraiss` ; puis `import tarifs` suivi de `tarifs.frai("Yoff", 2)`. Messages : `ModuleNotFoundError: No module named 'tarif'` (aucun fichier de ce nom) ; `ImportError: cannot import name 'fraiss' from 'tarifs' (...). Did you mean: 'frais'?` (le module existe, pas ce nom) ; `AttributeError: module 'tarifs' has no attribute 'frai'. Did you mean: 'frais'?`. Les versions récentes de Python proposent souvent le bon nom. Annulez.

3. `if __name__ == "__main__":`

Vous écrivez cette ligne depuis le chapitre 1. Voici enfin à quoi elle sert. Créez `convertir.py` :

```
def en_euros(montant_fcfa):
    return round(montant_fcfa / 655.957, 2)


print("__name__ vaut", __name__)
if __name__ == "__main__":
    print("Test :", en_euros(10000))
```

Lancez-le directement (`python convertir.py`) :

```
__name__ vaut __main__
Test : 15.24
```

Puis créez `essai3.py`, qui l'importe :

```
import convertir

print(convertir.en_euros(2500))
```

```
__name__ vaut convertir
3.81
```

Situation	Que vaut `__name__` dans `convertir.py` ?	Le test s'exécute-t-il ?
On **lance** le fichier : `python convertir.py`	`"__main__"` (« le programme principal »)	Oui
Un autre fichier l'**importe** : `import convertir`	`"convertir"`, le nom du module	Non

`__name__` est une variable que Python remplit tout seul dans chaque fichier. Grâce au test `if __name__ == "__main__":`, un même fichier peut servir de **module** (on importe ses fonctions, sans effet de bord) et de **programme** (on le lance pour l'essayer). **Règle** : dans un module, rien au premier niveau à part des importations, des constantes et des définitions ; tout le reste dans `main()`, appelée sous `if __name__ == "__main__":`.

Dans `tarifs.py`, supprimez maintenant le `print("chargement du module tarifs")` : il n'avait rien à faire au premier niveau.

4. Où Python cherche les modules

4.1 Le chemin de recherche

Quand vous écrivez `import tarifs`, Python cherche dans une liste de dossiers, dans l'ordre : c'est `sys.path`. Le **premier** dossier est celui du fichier lancé. Viennent ensuite les dossiers de la bibliothèque standard (`csv`, `json`, `random`...), puis ceux des paquets installés avec `pip` (section 6).

Deux conséquences :

1. Vos modules doivent être **dans le même dossier** que le programme lancé (ou dans un paquet, section 5). Lancer `python essais-ch6/essai1.py` depuis un autre dossier marche, puisque c'est le dossier **du fichier** qui compte ; mais un fichier rangé ailleurs n'est pas trouvé : `ModuleNotFoundError`.
2. Un de vos fichiers peut **masquer** un module de Python s'il porte le même nom, puisque votre dossier passe en premier.

4.2 Le piège du nom déjà pris

Dans un nouveau dossier, créez un fichier nommé `random.py` (pour « essayer le module random ») :

```
import random
print(random.randint(1, 6))
```

```
AttributeError: module 'random' has no attribute 'randint' (consider renaming '...\random.py'
since it has the same name as the standard library module named 'random' and prevents importing
that standard library module)
```

`import random` a importé **votre** fichier `random.py` (le premier trouvé), qui n'a pas de fonction `randint`. Python 3.13 vous explique même la cause : « envisagez de renommer ce fichier, il porte le nom d'un module de la bibliothèque standard ». Les versions plus anciennes donnent seulement `module 'random' has no attribute 'randint'`, ce qui est bien plus déroutant.

Règle : ne nommez jamais un fichier comme un module de Python ou un paquet installé : `random.py`, `csv.py`, `json.py`, `math.py`, `logging.py`, `pandas.py`, `tabulate.py`… Choisissez des noms à vous (`essai_random.py`). Pensez aussi à supprimer le dossier `__pycache__` créé à côté, s'il y en a un (section 5.3).

4.3 L'importation circulaire

Deux modules qui s'importent **l'un l'autre** forment une boucle. Python commence à exécuter le premier, s'interrompt pour exécuter le second, qui redemande le premier… à moitié chargé. C'est l'**importation circulaire**, une erreur fréquente quand on découpe un programme. Vous la provoquerez et la corrigerez dans l'exercice 5. Le message, avec Python 3.12 :

```
ImportError: cannot import name 'format_prix' from partially initialized module 'commandes' (most
likely due to a circular import)
```

(« impossible d'importer ce nom depuis le module `commandes`, partiellement initialisé, très probablement à cause d'une importation circulaire »). ⚠️ Python 3.13 donne un message trompeur pour ce cas : `cannot import name 'format_prix' from 'commandes' (consider renaming ...)`, comme s'il s'agissait d'un nom déjà pris. Lisez alors la **trace** : elle montre la boucle, fichier après fichier (`lancer.py` importe `commandes`, qui importe `clients`, qui importe `commandes`).

La solution : ce que les deux modules partagent va dans un **troisième** module, que les deux importent, et qui n'importe ni l'un ni l'autre. Dessinez les flèches « qui importe qui » : elles ne doivent jamais faire de boucle.

5. Les paquets

5.1 Un dossier de modules

Quand les modules deviennent nombreux, on les range dans un **paquet** (*package*) : un dossier qui contient un fichier `__init__.py`. Dans `essais-ch6`, créez cette arborescence :

```
essais-ch6/
├── essai4.py
└── livraisons/
    ├── __init__.py
    ├── tarifs.py
    └── devis.py
```

`livraisons/__init__.py` :

```
"""Le paquet livraisons de Yonnee Express."""
VERSION = "0.1"
```

`livraisons/tarifs.py` :

```
TARIFS = {"Médina": 1000, "Plateau": 1500, "Yoff": 2000, "Parcelles": 2000, "Pikine": 2500}


def frais(quartier, poids):
    return TARIFS[quartier] + (500 if poids > 5 else 0)
```

`livraisons/devis.py` :

```
from .tarifs import frais


def devis(quartier, poids, nombre=1):
    return f"{nombre} colis pour {quartier} : {frais(quartier, poids) * nombre} F"


if __name__ == "__main__":
    print(devis("Pikine", 7, 3))
```

Et `essai4.py`, **à côté** du dossier `livraisons` :

```
import sys

import livraisons
from livraisons.devis import devis
from livraisons import tarifs

print(livraisons.VERSION)
print(devis("Yoff", 2, nombre=4))
print(tarifs.frais("Médina", 8))
print(sys.path[0])
```

```
0.1
4 colis pour Yoff : 8000 F
1500
...\essais-ch6
```

Morceau	Ce qu'il veut dire
`__init__.py`	Fait du dossier un paquet. Il est exécuté à la première importation du paquet ; il peut être vide, ou contenir ce qui concerne tout le paquet (ici `VERSION`)
`import livraisons`	Importe le paquet, c'est-à-dire son `__init__.py`
`from livraisons.devis import devis`	Le point sépare les niveaux : la fonction `devis` du module `devis` du paquet `livraisons`

Morceau	Ce qu'il veut dire
`from .tarifs import frais` (dans `devis.py`)	Importation **relative** : le point seul veut dire « dans **mon** paquet ». `devis.py` trouve `tarifs.py` à côté de lui, quel que soit le nom du paquet
`sys.path[0]`	Le premier dossier de recherche : celui de `essai4.py`. C'est pour cela que Python trouve le dossier `livraisons`

5.2 Lancer un module d'un paquet : `python -m`

Essayez de lancer le test de `devis.py` directement :

```
python livraisons/devis.py
```

```
ImportError: attempted relative import with no known parent package
```

Lancé comme un simple fichier, `devis.py` ne sait pas qu'il fait partie d'un paquet : le point de `from .tarifs` ne veut plus rien dire. La bonne façon, depuis le dossier `essais-ch6` :

```
python -m livraisons.devis
```

```
3 colis pour Pikine : 9000 F
```

`python -m livraisons.devis` (*m* pour module) dit : « lance le module `devis` **du paquet** `livraisons` ». On écrit des points et pas de `.py`, comme dans un `import`.

5.3 Le dossier `__pycache__`

Après une importation, un dossier `__pycache__` apparaît en général dans le dossier des modules. Python y range une version déjà traduite de chaque module (des fichiers `.pyc`), pour démarrer plus vite la fois suivante. On peut le supprimer sans risque (il se recrée), et on ne le met **pas** sur GitHub : ajoutez `__pycache__/` à votre `.gitignore`.

6. Les paquets externes : environnement virtuel et `pip`

6.1 Bibliothèque standard et paquets externes

Python est livré avec une **bibliothèque standard** (`csv`, `json`, `datetime`, `collections`, `logging`…) : rien à installer. Des centaines de milliers d'autres paquets sont publiés par la communauté sur **PyPI** (*Python Package Index*, pypi.org) : `pandas`, `requests`, `flask`… On les installe avec `pip`.

Le problème : si chaque projet installe ses paquets dans le **même** Python de l'ordinateur, les projets finissent par se gêner (le projet A veut `pandas` 2, le projet B `pandas` 3), et on ne sait plus de quoi chaque projet a besoin. La solution : un **environnement virtuel** par projet.

🟢 Un environnement virtuel est un **Python privé**, rangé dans un dossier du projet (`.venv`), avec ses propres paquets. Ce qu'on y installe n'existe que là.

6.2 Créer et utiliser un environnement virtuel

Dans le dossier de votre projet, dans le terminal :

```
python -m venv .venv
```

Cette commande crée le dossier `.venv` , qui contient une copie de Python (`.venv/Scripts/python.exe` sous Windows, `.venv/bin/python` sous Linux et macOS) et son propre `pip` . Puis on **active** l'environnement, pour que `python` et `pip` désignent ceux du projet :

Système	Commande d'activation
Windows, PowerShell (terminal par défaut de VS Code)	`.venv\Scripts\Activate.ps1`
Windows, invite de commandes (cmd)	`.venv\Scripts\activate.bat`
Linux, macOS	`source .venv/bin/activate`

Le nom `(.venv)` apparaît au début de la ligne du terminal : l'environnement est actif. `deactivate` le désactive.

⚠️ **PowerShell refuse parfois d'exécuter le script d'activation** (message : l'exécution de scripts est désactivée sur ce système). Deux solutions sans toucher aux réglages de sécurité de Windows : ouvrir un terminal **cmd** dans VS Code (flèche à côté du `+` du terminal, puis « Command Prompt ») et utiliser `activate.bat` ; ou, plus simple, laisser VS Code faire : `Ctrl+Shift+P` , « Python: Select Interpreter », choisir celui de `.venv` . Les nouveaux terminaux s'ouvriront avec l'environnement actif.

6.3 Installer un paquet

Le paquet `tabulate` affiche des listes sous forme de jolis tableaux. Environnement actif :

```
pip install tabulate
```

Puis, dans un fichier `essai_tableau.py` (pas `tabulate.py` ! section 4.2) :

```
from tabulate import tabulate

print(tabulate([["Pikine", 1076000], ["Yoff", 967000]], headers=["quartier", "frais"],
         tablefmt="github"))
```

```
| quartier   |   frais |
|------------|---------|
| Pikine     | 1076000 |
| Yoff       |  967000 |
```

Désactivez l'environnement (`deactivate`) et relancez : `ModuleNotFoundError: No module named 'tabulate'` . Le paquet n'existe **que** dans l'environnement du projet : c'est voulu.

Commande	Ce qu'elle fait
`pip install tabulate`	Télécharge le paquet depuis PyPI et l'installe dans l'environnement actif
`pip install tabulate==0.10.0`	Une version précise
`pip list`	Les paquets installés dans l'environnement
`pip uninstall tabulate`	Désinstalle
`pip freeze > requirements.txt`	Écrit la liste exacte des paquets et de leurs versions dans le fichier `requirements.txt`
`pip install -r requirements.txt`	Installe tout ce que liste `requirements.txt`

Si une ligne `A new release of pip is available` s'affiche, ce n'est pas une erreur : une nouvelle version de `pip` existe, vous pouvez l'ignorer.

6.4 Partager le projet : `requirements.txt`

Le dossier `.venv` pèse vite des dizaines de mégaoctets, et il dépend de votre ordinateur : on ne le met **jamais** sur GitHub. On partage à la place la **liste** des paquets :

```
pip freeze > requirements.txt
```

```
tabulate==0.10.0
```

Quelqu'un qui récupère votre projet refait l'environnement en trois commandes : `python -m venv .venv` , activer, `pip install -r requirements.txt` . Ajoutez `.venv/` à votre `.gitignore` , et `requirements.txt` à votre dépôt.

🟣 Vous verrez après la POO des outils plus modernes qui font tout cela à la fois (`uv` , le fichier `pyproject.toml`). Les idées restent les mêmes : un environnement isolé par projet, et une liste des dépendances avec leurs versions.

7. Le journal : `logging`

7.1 Pourquoi pas `print` ?

Dans les ateliers, vous affichez avec `print` à la fois les **résultats** (le total des frais) et des **messages de fonctionnement** (« ligne 6 rejetée », « fichier introuvable », les durées du chronomètre). Ce mélange pose problème : impossible de faire taire les messages de fonctionnement sans modifier le code, impossible de les garder dans un fichier pour les relire demain, impossible de distinguer un détail d'une erreur grave.

Le module `logging` (bibliothèque standard) tient un **journal** : chaque message a un **niveau** de gravité, et un seul réglage décide de ce qui s'affiche et où.

Niveau	Pour…	Exemple
`DEBUG`	les détails utiles au développeur	« ligne 2 lue », la durée d'une fonction
`INFO`	le déroulement normal	« lecture du registre : 60 lignes »
`WARNING`	quelque chose d'anormal, mais le programme continue	« ligne 6 rejetée », « tarifs par défaut »
`ERROR`	une opération a échoué	« conversion impossible »
`CRITICAL`	le programme ne peut pas continuer	« registre introuvable »

7.2 Les niveaux et le réglage

Créez `essai5.py` :

```
import logging

logging.debug("détail pour le développeur")
logging.info("information")
logging.warning("attention : poids suspect")
logging.error("erreur : quartier inconnu")
```

```
WARNING:root:attention : poids suspect
ERROR:root:erreur : quartier inconnu
```

Sans réglage, le journal n'affiche qu'à partir de `WARNING` : `debug` et `info` sont **ignorés**. `root` est le nom du journal principal. (Avec un journal nommé, section suivante, et sans aucun réglage, les messages `WARNING` et plus graves s'affichent tels quels, sans le niveau ni le nom.) On règle le journal **une seule fois**, au début du programme, avec `basicConfig` . Créez `essai6.py` :

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)-8s %(name)s : %(message)s",
    datefmt="%H:%M:%S",
)
journal = logging.getLogger("yonnee.lecture")

journal.debug("ligne 2 lue")
journal.info("lecture du registre : 60 lignes")
journal.warning("ligne 6 rejetée : frais incohérents")
try:
    int("deux")
except ValueError:
    journal.exception("conversion impossible")
```

```
23:50:35 INFO     yonnee.lecture : lecture du registre : 60 lignes
23:50:35 WARNING  yonnee.lecture : ligne 6 rejetée : frais incohérents
23:50:35 ERROR    yonnee.lecture : conversion impossible
Traceback (most recent call last):
  File "...\essai6.py", line 14, in <module>
    int("deux")
ValueError: invalid literal for int() with base 10: 'deux'
```

Morceau	Ce qu'il veut dire
`level=logging.INFO`	Le seuil : les messages `INFO` et plus graves s'affichent, `DEBUG` est ignoré. Pour tout voir en cherchant une erreur : `logging.DEBUG`
`format=...`	La forme de chaque message : `%(asctime)s` l'heure, `%(levelname)-8s` le niveau sur 8 caractères, `%(name)s` le nom du journal, `%(message)s` le message
`datefmt="%H:%M:%S"`	L'heure seule (les mêmes codes que `strftime`). L'heure affichée est bien sûr celle de votre exécution
`logging.getLogger("yonnee.lecture")`	Un journal **nommé**. Dans un module, on écrit `journal = logging.getLogger(__name__)` : le nom du module, par exemple `yonnee.lecture`, apparaît dans chaque message et dit d'où il vient
`journal.exception(...)`	Dans un `except` : un message `ERROR` **suivi de la trace** de l'exception. Le programme continue

Qui règle, qui écrit ? Les **modules** se contentent d'écrire dans leur journal (`journal.warning(...)`) ; seul le **programme principal** appelle `basicConfig`. Ainsi, le même module peut être bavard dans un programme et silencieux dans un autre.

🔵 Pour un message avec des valeurs, `logging` préfère `journal.warning("ligne %d rejetée : %s", numero, raison)` à une f-string : le texte n'est fabriqué que si le message est effectivement affiché. `%d` attend un entier, `%s` n'importe quelle valeur. Les deux écritures marchent.

7.3 Un journal dans un fichier

Créez `essai7.py` :

```python
import logging

logging.basicConfig(
    filename="journal.log",
    encoding="utf-8",
    level=logging.DEBUG,
    format="%(asctime)s %(levelname)s %(message)s",
)
logging.debug("démarrage")
logging.warning("ligne 13 rejetée : livreur vide")
print("Le programme continue ; le journal est dans journal.log")
```

```
Le programme continue ; le journal est dans journal.log
```

L'écran n'affiche que le `print` ; les messages sont dans `journal.log` :

```
2026-10-23 23:50:36,282 DEBUG démarrage
2026-10-23 23:50:36,282 WARNING ligne 13 rejetée : livreur vide
```

Lancez le programme une deuxième fois : le fichier a maintenant **4** lignes. Par défaut, le journal **ajoute** à la fin du fichier, ce qui garde l'historique des exécutions. Pour repartir d'un fichier vide à chaque fois : `filemode="w"`. N'oubliez pas `encoding="utf-8"` pour les accents (chapitre 2), et ajoutez `*.log` à votre `.gitignore`.

🟣 Écran **et** fichier à la fois, avec des seuils différents : on crée deux « destinations » (*handlers*) et on les donne à `basicConfig` :

```
console = logging.StreamHandler()
console.setLevel(logging.INFO)
fichier = logging.FileHandler("journal.log", mode="w", encoding="utf-8")
fichier.setLevel(logging.DEBUG)
logging.basicConfig(level=logging.DEBUG, handlers=[console, fichier], format="%(levelname)s %
        (name)s : %(message)s")
```

Les messages `INFO` et plus graves s'affichent ; **tout**, `DEBUG` compris, va dans le fichier. C'est ce que fera l'outil de l'atelier.

`print`	`logging`
Les **résultats** destinés à l'utilisateur (un total, un tableau)	Le **fonctionnement** du programme (lectures, rejets, durées, erreurs)
Toujours affiché	Affiché ou non selon le niveau, à l'écran ou dans un fichier

8. Vers la programmation orientée objet

Regardez ce que contiendra votre paquet à la fin de l'atelier : un module `modele.py` avec le tuple nommé `Livraison` et la fonction `frais_livraison(tarifs, quartier, poids)` ; un module `lecture.py` avec `convertir(ligne, tarifs)` qui vérifie une livraison ; des filtres qui reçoivent une livraison… Les **données** (une livraison) d'un côté, les **fonctions** qui les concernent de l'autre, et le même paramètre `tarifs` qui passe de fonction en fonction.

Trois limites apparaissent :

1. Rien n'empêche de créer une `Livraison` avec un poids négatif ou des frais faux : la vérification est dans une autre fonction, qu'on peut oublier d'appeler.
2. Pour savoir si une livraison est en retard ou lourde, il faut savoir **quelle fonction** appeler et dans **quel module**.

3. Un tuple nommé ne peut pas changer (impossible de corriger un statut « livré »), et il n'a pas de comportement à lui.

La **programmation orientée objet** répond à ces trois limites en réunissant, dans une même **classe**, les données **et** les opérations qui les concernent : une livraison qui **se vérifie** à sa création, qui **sait** calculer ses frais et dire si elle est en retard. C'est le chapitre 7, et l'atelier 7 réécrira ce paquet avec des classes.

Exercices

Créez un dossier `exercices-ch6`, avec un sous-dossier par exercice.

- ★ **Exercice 1 : un module de géométrie.** Écrivez `geometrie.py` avec une constante `PI` (prise dans le module `math`), `aire_cercle(rayon)` et `perimetre_rectangle(longueur, largeur)`, et un bloc de tests sous `if __name__ == "__main__":` qui affiche `True` si `aire_cercle(1)` arrondie à 4 chiffres vaut 3,1416 et si `perimetre_rectangle(3, 2)` vaut 10. Puis `utiliser.py`, qui l'importe des trois façons de la section 2.2 et affiche l'aire d'un cercle de rayon 2 (attendu : 12,57), le périmètre d'un rectangle de 12,5 sur 8 (41,0) et `PI` à 5 décimales. Vérifiez que les tests ne s'affichent **pas** quand on lance `utiliser.py`. (Sections 2 et 3.)
- ★ **Exercice 2 : un paquet `ecole`.** Créez le paquet `ecole` avec `__init__.py`, `notes.py` (`moyenne(notes)` arrondie à 2 décimales ; `mention(moy)` : Très bien à partir de 16, Bien à partir de 14, Assez bien à partir de 12, Passable à partir de 10, sinon Ajourné) et `bulletin.py`, qui importe les deux fonctions avec une importation **relative** et définit `bulletin(nom, notes)`, qui renvoie par exemple `Mariama : 13.0 / 20, Assez bien`. Ajoutez un test dans `bulletin.py`, lancez-le avec `python -m`, puis directement : quel message obtenez-vous, et pourquoi ? Enfin, un `principal.py` à côté du paquet affiche le bulletin de deux étudiants. (Section 5.)
- ★★ **Exercice 3 : un environnement virtuel.** Dans le dossier de l'exercice, créez un environnement virtuel, activez-le, installez `tabulate`, et affichez sous forme de tableau trois étudiants avec leur moyenne et leur mention (vous pouvez réutiliser les résultats de l'exercice 2). Produisez le `requirements.txt`. Désactivez l'environnement et relancez : quel message ? Écrivez le `.gitignore` du dossier. (Section 6.)
- ★★ **Exercice 4 : un journal de conversion.** Écrivez `en_fahrenheit(celsius)` (°C x 9 / 5 + 32, arrondi à 0,1). La fonction écrit dans un journal nommé `temperatures` : un message `DEBUG` à chaque conversion ; un `WARNING` au-delà de 60 °C (valeur suspecte pour une température d'air) ; un `ERROR` sous -273,15 °C (sous le zéro absolu), suivi d'une `ValueError`. Le journal va dans le fichier `temperatures.log`, recréé à chaque exécution. Testez avec 25, 41,5, 75 et -300 : qu'affiche l'écran, que contient le fichier ? (Section 7.)
- ★★★ **Exercice 5 : la boucle d'importation.** Une petite boutique a découpé son programme en trois fichiers. `commandes.py` importe `nom_client` de `clients.py` et définit `resume(commande)`, qui renvoie par exemple `commande 7 de Babacar`, ainsi que `format_prix(montant)`, qui renvoie `"12500 F"`. `clients.py` importe `format_prix` de `commandes.py` et définit un dictionnaire `CLIENTS = {1: "Aissatou", 2: "Babacar"}`, `nom_client(numero)` et `solde(numero, montant)`, qui renvoie `Aissatou doit 12500 F`. `lancer.py` importe `resume` et affiche `resume({"id": 7, "client": 2})`. Écrivez ces

trois fichiers, lancez `lancer.py` et lisez la trace : quelle boucle montre-t-elle ? Corrigez l'organisation (indice : section 4.3) pour que `lancer.py` affiche le résumé de la commande et le solde d'Aissatou. Dessinez les flèches d'importation avant et après. (Section 4.)

Atelier 6 : l'outil complet en modules

Le travail principal du chapitre : vous réunissez le meilleur des ateliers 2 à 5 dans un **paquet** `yonnee`, un module par rôle (le modèle, la lecture avec ses rejets, les filtres, les rapports, les outils), avec un programme principal `main.py`, un environnement virtuel, un `requirements.txt` et un journal à l'écran et dans un fichier. Le résultat doit redonner **exactement** les chiffres des ateliers précédents. L'énoncé vous est fourni séparément.

A retenir

- Un **module** est un fichier `.py` ; l'importer l'**exécute** une seule fois. `import m` (le cas général), `from m import nom`, `import m as court` ; jamais `from m import *`.
- `if __name__ == "__main__":` : le code qui ne doit tourner que si l'on **lance** le fichier ; `__name__` vaut `"__main__"` quand on le lance, le nom du module quand on l'importe.
- Python cherche les modules dans `sys.path`, en commençant par le dossier du fichier lancé : ne jamais nommer un fichier comme un module existant (`random.py`, `csv.py`…).
- Pas de boucle d'importation : ce qui est partagé va dans un troisième module.
- Un **paquet** est un dossier avec `__init__.py` ; `from .module import nom` dans le paquet ; on lance un module d'un paquet avec `python -m paquet.module`.
- Un **environnement virtuel** par projet : `python -m venv .venv`, activer, `pip install ...`, `pip freeze > requirements.txt` ; `.venv/`, `__pycache__/` et `*.log` dans le `.gitignore`.
- `print` pour les résultats, `logging` pour le fonctionnement ; niveaux `DEBUG`, `INFO`, `WARNING`, `ERROR`, `CRITICAL` ; `getLogger(__name__)` dans les modules, `basicConfig` une seule fois dans le programme principal.

Erreurs fréquentes

Message ou symptôme	Ce que ça veut dire	Solution
`ModuleNotFoundError: No module named 'tarif'`	Aucun fichier ni paquet de ce nom dans `sys.path`	Vérifier l'orthographe, le dossier du fichier lancé
`ModuleNotFoundError: No module named 'tabulate'`	Paquet externe non installé, ou environnement virtuel non actif	Activer `.venv`, puis `pip install -r requirements.txt`
`ImportError: cannot import name 'fraiss' from 'tarifs'`	Le module existe, pas ce nom	Vérifier le nom dans le module (Python suggère souvent le bon)

Message ou symptôme	Ce que ça veut dire	Solution
`AttributeError: module 'random' has no attribute 'randint'`	Un de vos fichiers s'appelle `random.py` et masque le vrai module	Renommer le fichier, supprimer `__pycache__`
`ImportError: ... partially initialized module ... circular import` (ou `consider renaming` avec Python 3.13)	Deux modules s'importent l'un l'autre	Lire la trace, déplacer le code partagé dans un troisième module
`ImportError: attempted relative import with no known parent package`	Module d'un paquet lancé directement comme un fichier	`python -m paquet.module` depuis le dossier qui contient le paquet
Un module affiche des choses quand on l'importe	Du code au premier niveau du module	Le mettre dans une fonction, sous `if __name__ == "__main__":`
PowerShell refuse `Activate.ps1`	Exécution de scripts désactivée	Terminal cmd et `activate.bat`, ou choisir l'interpréteur `.venv` dans VS Code
`logging.info(...)` n'affiche rien	Seuil par défaut à `WARNING`	`logging.basicConfig(level=logging.INFO)` au début du programme principal
`basicConfig` semble ignoré	Il n'agit qu'une fois : un premier message ou un premier `basicConfig` a déjà réglé le journal	L'appeler tout au début de `main()`, avant tout message
Le fichier journal grossit à chaque exécution	Mode ajout par défaut	`filemode="w"` (ou `mode="w"` pour un `FileHandler`)

Pour aller plus loin

- Le tutoriel officiel, « Modules » (en français) : https://docs.python.org/fr/3/tutorial/modules.html
- Le tutoriel officiel, « Environnements virtuels et paquets » (en français) : https://docs.python.org/fr/3/tutorial/venv.html
- Le tutoriel officiel de `logging` (en français) : https://docs.python.org/fr/3/howto/logging.html
- PyPI, l'index des paquets Python : https://pypi.org